LAPORAN UTS SERVICE

Nama : Putri Calista Devianof
NIM : 2410511141
Kelas : B

I. PENDAHULUAN

A. Latar Belakang

Sistem pemesanan tiket event merupakan salah satu aplikasi yang membutuhkan skalabilitas tinggi karena volume transaksi bisa meningkat drastis saat event populer dibuka pendaftarannya. Pendekatan monolitik yang menempatkan semua fitur dalam satu aplikasi besar akan kesulitan menghadapi lonjakan traffic ini karena seluruh sistem harus disesuaikan kembali ukurannya sekaligus meski hanya satu fitur lah yang memiliki beban tinggi.

B. Tujuan

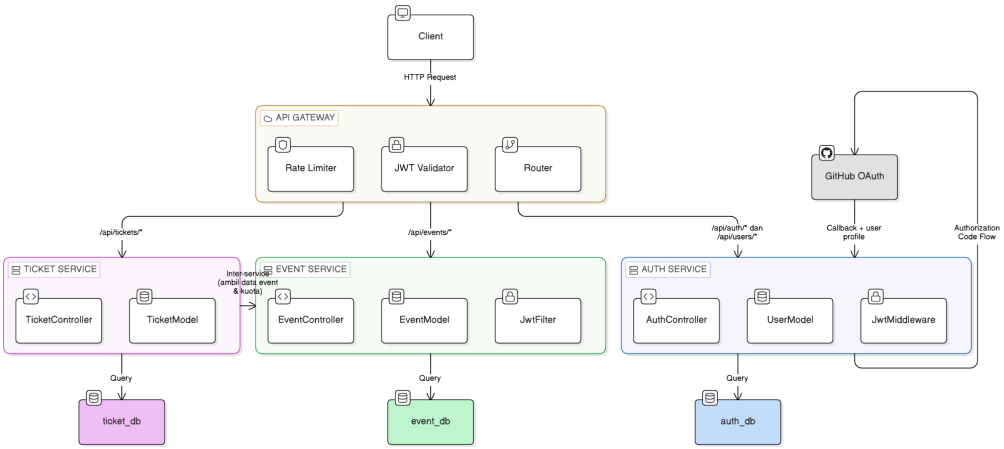
- Membangun sistem pemesanan tiket event menggunakan arsitektur microservice
- Mengimplementasikan autentikasi JWT dan OAuth 2.0 Github
- Membangun API Gateway sebagai single entry point
- Mengimplementasikan komunikasi antar service melalui REST API

C. Ruang Lingkup

Sistem yang dibangun mencangkup fitur daftar event, kategori tiket, checkout, tiket dengan QR code, hingga validasi tiket masuk.

II. ARSITEKTUR SISTEM

A. Gambaran Umum



Sistem terdiri dari 4 komponen utama yang berjalan sebagai container Docker terpisah:

- API Gateway berjalan di port 8000, menjadi satu-satunya pintu masuk semua request dari client

- Auth Service berjalan di port 8001, menangani autentikasi dan manajemen user
- Event Service berjalan di port 8002, menangani manajemen event dan kategori tiket
- Tiket Service berjalan di port 8003, menangani pemesanan tiket dan validasi

B. Peta Routing Gateway

Semua request masuk melalui API Gateway di port 8000. Gateway memvalidasi JWT terlebih dahulu sebelum meneruskan request ke service yang sesuai berdasarkan path prefix:

- Path /api/auth/* diteruskan ke Auth Service di port 8001
- Path /api/users/* diteruskan ke Event Service di port 8001
- Path /api/events/* diteruskan ke Event Service di port 8002
- Path /api/tickets/* diteruskan ke Tiket Service di port 8003

Sehingga client tidak pernah mengakses service secara langsung. Semua komunikasi wajib melalui gateway.

C. Justifikasi Pemisahan Service

Keputusan menggunakan arsitektur microservice didasarkan pada beberapa pertimbangan teknis berikut.

Pada pendekatan monolitik, seluruh fitur yakni autentikasi, manajemen event, dan, pemesanan tiket, berada dalam satu aplikasi besar. Ketika terjadi lonjakan traffic pada fitur pemesanan tiket saat event populer dibuka, seluruh aplikasi harus melakukan pengukuran ulang padahal fitur autentikasi dan manajemen event tidak membutuhkan resource tambahan. Selain itu, satu bug kritis di salah satu fitur bisa mematikan seluruh sistem.

Dengan pendekatan microservice yang dipilih, setiap service dapat di-scale secara independen sesuai kebutuhan. Ketika Ticket Service mengalami lonjakan traffic, hanya ticket service yang perlu di-scale tanpa memengaruhi Auth Service dan Event Service. Kegagalan pada satu service juga akan terisolasi, sehingga bila misalnya Event Service down, maka tetap bisa melayani login dan OAuth.

Pemisahan Auth Service menjadi service tersendiri didasarkan pada alasan bahwa autentikasi adalah fitur lintas layanan yang dikonsumsi oleh semua service lain. Dengan memisahkannya perubahan pada logika autentikasi tidak memengaruhi kode Event Service atau Ticket Service.

Pemisahan Event Service menggunakan PHP CodeIgniter 4 didasarkan juga pada kebutuhan untuk menggunakan teknologi yang paling sesuai per domain. Event Service banyak melakukan operasi database relasional yang kompleks dengan join antar tabel dan CodeIgniter 4 dengan pola MVC menyediakan struktur yang bersih untuk kebutuhan ini.

Pemisahan Ticket Service didasarkan pada alasan bahwa pemesanan tiket memiliki logika bisnis yang kompleks dan berbeda dari manajemen event, mulai dari pengecekan kuota, generate kode unik, generate qr code, hingga validasi tiket. Memisahkannya memungkinkan pengembangan dan pengujian fitur ini secara independen.

III. IMPLEMENTASI

A. Tech Stack

- API Gateway : Node.js Express dengan library http-proxy-middleware
- Auth Service : Node.js Express dengan MySQL
- Event Service : PHP CodeIgniter 4 dengan Apache dan MySQL
- Ticket Service : Node.js Express dengan MySQL
- Database : MySQL 8.0 untuk semua service
- QR Code : library qrcode untuk Node.js
- Containerisasi : Docker dan Docker Compose

B. Auth Service

Auth service dibangin menggunakan Node.js Express dan bertanggung jawab atas seluruh proses autentikasi sistem. Service ini menggunakan database auth_db yang memiliki 4 tabel:

- Tabel users menyimpan data semua user baik yang mendaftar manual maupun melalui OAuth
- Tabel oauth_accounts menyimpan data akun Oauth yang terhubung ke user lokal
- Tabel refresh_tokens menyimpan refresh token yang aktif per user
- Tabel token_blacklist menyimpan access token yang sudah di logout

```
mysql> SHOW TABLES;
+-------------------+
| Tables_in_auth_db |
+-------------------+
| oauth_accounts    |
| refresh_tokens    |
| token_blacklist   |
| users             |
+-------------------+
4 rows in set (0.01 sec)

mysql> DESCRIBE users;
+----------------+---------------------+------+-----+-------------------+-----------------------------------------------+
| Field          | Type                | Null | Key | Default           | Extra                                         |
+----------------+---------------------+------+-----+-------------------+-----------------------------------------------+
| id             | int                 | NO   | PRI | NULL              | auto_increment                                |
| nama           | varchar(100)        | NO   |     | NULL              |                                                |
| email          | varchar(100)        | NO   | UNI | NULL              |                                                |
| password       | varchar(255)        | YES  |     | NULL              |                                                |
| role           | enum('user','admin') | YES  |     | user              |                                                |
| foto_profil    | varchar(255)        | YES  |     | NULL              |                                                |
| oauth_provider | varchar(50)         | YES  |     | NULL              |                                                |
| is_aktif       | tinyint(1)          | YES  |     | 1                 |                                                |
| dibuat_pada    | timestamp           | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED                              |
| diubah_pada    | timestamp           | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED on update CURRENT_TIMESTAMP |
+----------------+---------------------+------+-----+-------------------+-----------------------------------------------+
10 rows in set (0.06 sec)

mysql> DESC oauth_accounts;
+----------------+-------------+------+-----+-------------------+-------------------+
| Field          | Type        | Null | Key | Default           | Extra             |
+----------------+-------------+------+-----+-------------------+-------------------+
| id             | int         | NO   | PRI | NULL              | auto_increment    |
| user_id        | int         | NO   | MUL | NULL              |                   |
| provider       | varchar(50) | NO   | MUL | NULL              |                   |
| provider_id    | varchar(100) | NO   |     | NULL              |                   |
| provider_email | varchar(100) | YES  |     | NULL              |                   |
| provider_nama  | varchar(100) | YES  |     | NULL              |                   |
| foto_profil    | varchar(255) | YES  |     | NULL              |                   |
| dibuat_pada    | timestamp   | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+----------------+-------------+------+-----+-------------------+-------------------+
8 rows in set (0.00 sec)
```

C. Implementasi JWT

Sistem menggunakan JSON Web Token untuk autentikasi. Setiap user yang berhasil login mendapatkan 2 token, yakni access token yang berlaku 15 menit dan refresh token yang berlaku 7 hari.

Validasi JWT tidak dilakukan secara manual di setiap endpoint. Sistem menggunakan middleware yang diterapkan secara global dengan authMiddleware.js di gateway untuk semua request, dan jwtMiddleware.js di masing-masing service untuk endpoint yang membutuhkan autentikasi.

Saat logout, access token dimasukkan ke tabel token_blacklist sehingga tidak bisa dipakai lagi meskipun belum expired. Refresh token dihapus pula dari refresh_tokens.

D. Implementasi OAuth 2.0 Github

Sistem mengimplementasikan login alternatif menggunakan Github Oauth dengan Authorization Code Flow, bukan Implicit Flow yang sudah deprecated. Alur implementasinya adalah sebagai berikut:

Pertama, user mengakses endpoint GET /api/auth/oauth/github. Sistem langsung redirect ke halaman login GitHub resmi dengan parameter client_id, redirect_url, dan scope.

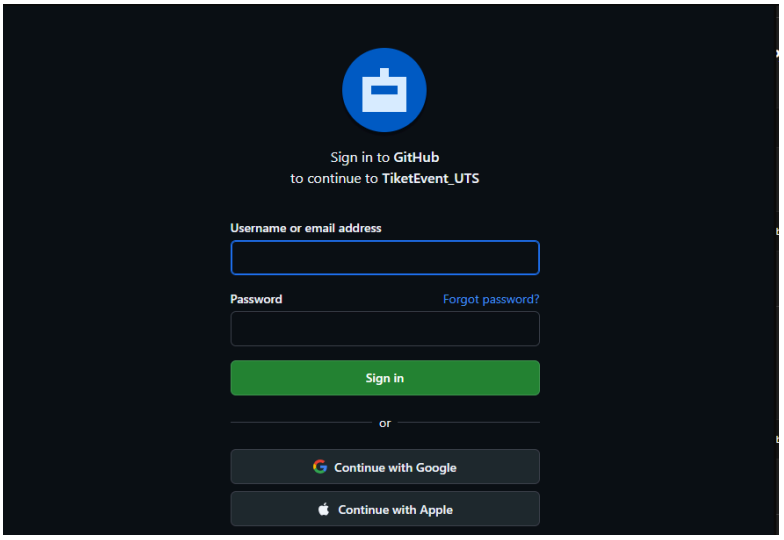
Kedua, setelah user login dan approve di Github, Github mengirimkan authorization code ke callback URL sistem.

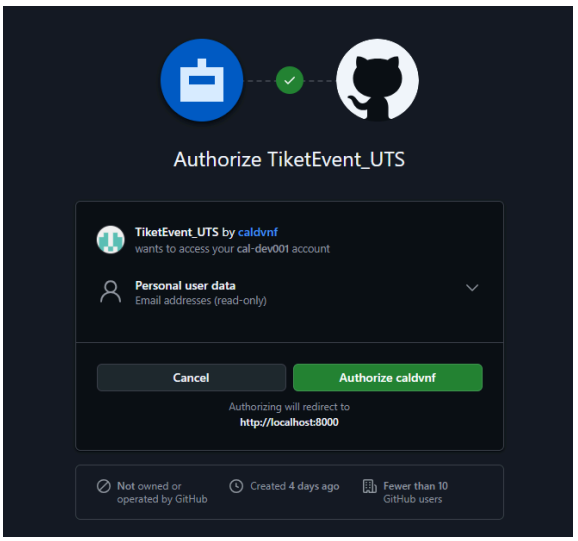
Ketiga, sistem menukar authorization code tersebut dengan access token Github melalui request endpoint Github OAuth.

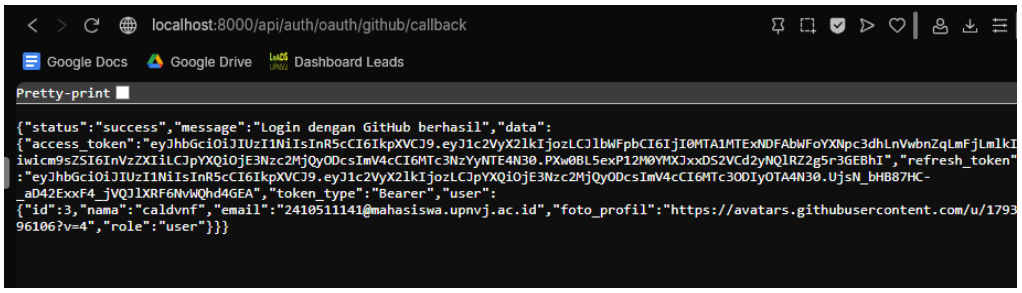
Keempat, menggunakan access token Github, sistem mengambil data profil user, yakni nama, email, dan foto profil.

Kelima, sistem memeriksa apakah user sudah pernah login GitHub sebelumnya dengan memeriksa tabel oauth_accounts. Kalau sudah ada, sistem mengambil data user lokal yang terhubung. Kalau belum ada, sistem membuat akun user baru secara otomatis dengan field oauth_provider yang terisi oleh github.

Keenam, sistem menghasilkan JWT token untuk user tersebut, sama seperti login biasa.







E. Event Service

Event Service dibangun menggunakan PHP CodeIgniter dengan pola MVC sesuai requirement. Pemisahan layer dilakukan sebagai berikut:

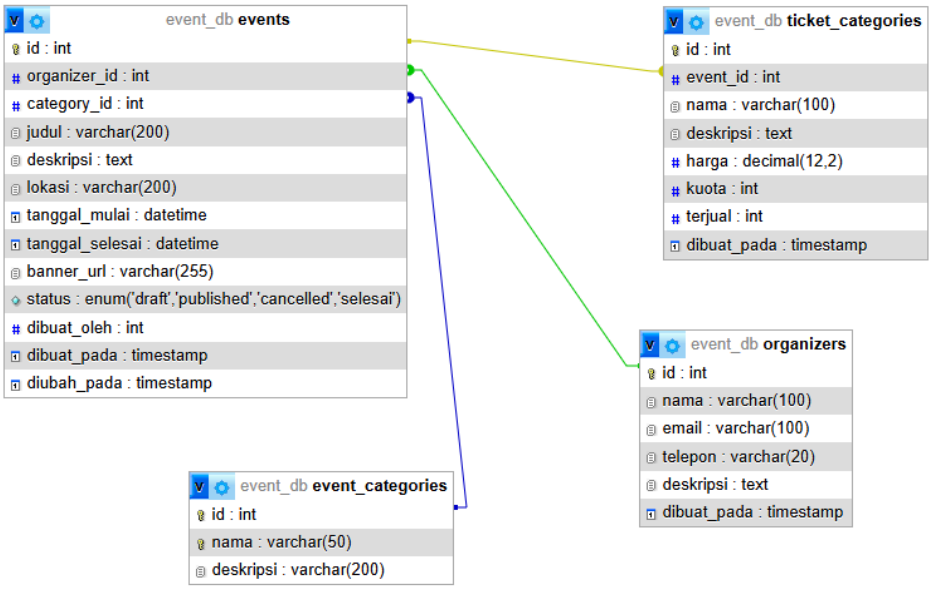
- Layer Model berada di folder app/Models, berisi semua query database. EventModel.php menangani query untuk tabel events, dan TicketCategoryModel.php menangani query untuk tabel ticket_categories
- Layer Controller berada di folder app/Controllers, berisi logika bisnis. EventController.php menangani semua request yang masuk dan memanggil Model untuk operasi database
- Layer Route berada di file app/Config/Routes.php, berisi pendaftaran semua endpoint

Tidak ada logika bisnis atau query database yang ditulis langsung di file route. Ini memenuhi requirement soal yang melarang semua logika ditulis dalam satu file.

Event service juga mengimplementasikan inter-service communication, dimana endpoint internal /events/internal/kurangi-kuota dipanggil oleh Ticket Service setiap kali ada pemesanan tiket untuk mengurangi kuota yang tersedia.

Database event_db memiliki 4 tabel yang saling berelasi:

- Tabel organizers menyimpan data penyelenggara event
- Tabel event_categories menyimpan master kategori event seperti Konser, Seminar, Workshop
- Tabel events menyimpan data utama event dengan foreign key ke organizers dan event_categories
- Tabel ticket_categories menyimpan kategori tiket per event seperti regular dan VIP dengan foreign key ke event

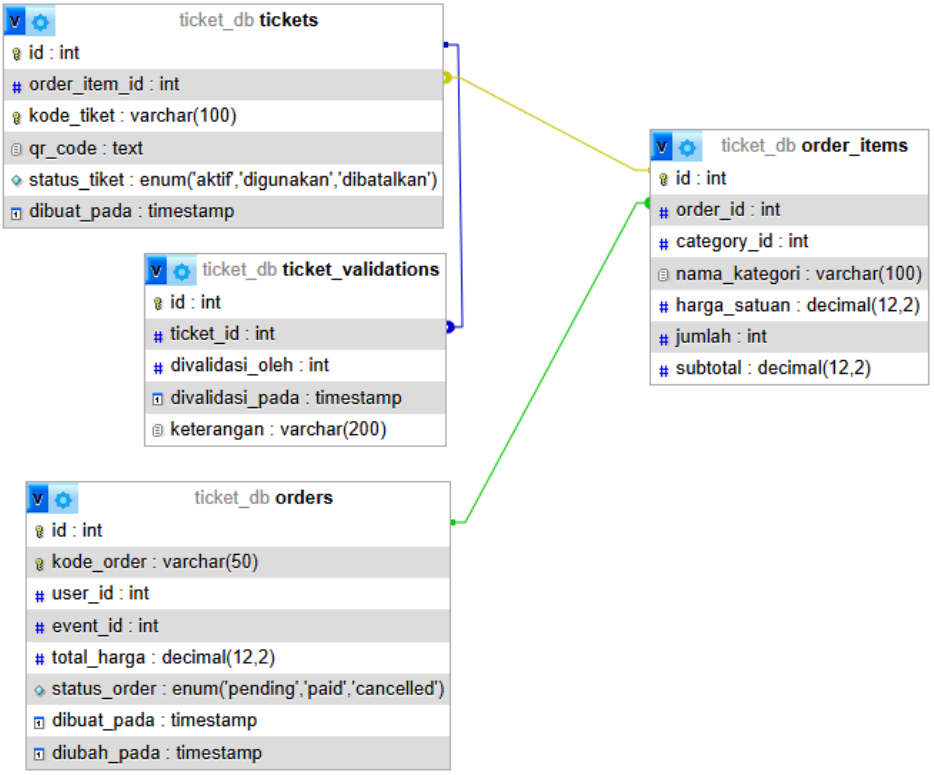


Endpoint GET /api/events mengimplementasikan paging dan filtering sesuai requirement soal. Paging menggunakan parameter page dan per-page. Filtering tersedia berdasarkan status event, kata kunci judul, lokasi, dan kategori event.

F. Ticket Service

Ticket Service dibangun menggunakan Node.js Express dan menangani seluruh proses pemesanan tiket. Database ticket_db memiliki 4 tabel yang saling berelasi:

- Tabel orders menyimpan data order dengan status pending, paid, atau canceled
- Tabel order_items menyimpan detail ticket yang dipesan dalam satu order
- Tabel tickets menyimpan tiket individual hasil dari order beserta QR code dalam format base64
- Tabel ticket validations menyimpan log validasi tiket



Saat user memesan tiket, Ticket Service melakukan komunikasi ke Event Service untuk mengambil data harga dan mengecek ketersediaan kuota. Ini adalah implementasi inter-service communication dimana Ticket service memanggil endpoint Event Service secara langsung menggunakan library axios.

QR Code di generate menggunakan library qrcode dan disimpan dalam format base64 di database. QR code berisi kode tiket unik yang di generate menggunakan UUID.

G. API Gateway

API Gateway dibangun menggunakan Node.js Express dengan library http-proxy-middleware. Gateway berfungsi sebagai single entry point yang menerima semua request dari client dan meneruskannya ke service yang sesuai.

Gateway mengimplementasikan tiga fitur utama sesuai requirement soal:

Pertama, routing berdasarkan path prefix ke service yang sesuai. Request dengan path /api/auth/* dan /api/users/* diteruskan ke Auth Service, /api/events/* ke Event Service, dan /api/tickets/* ke Ticket Service.

Kedua, validasi JWT di sisi gateway sebelum request diteruskan. Semua endpoint kecuali register, login, refresh, dan OAuth tidak memerlukan token. Kalau token tidak valid atau tidak ada, gateway langsung menolak request dengan response 401 tanpa perlu meneruskan ke service.

Ketiga, rate limiting menggunakan library express-rate-limit yang membatasi maksimal 60 request per menit per IP address. Kalau melebihi batas, gateway menolak request dengan response 429 Too Many Requests.

IV. PENGUJIAN

A. Hasil Pengujian Endpoint

Pengujian menggunakan Postman. Berikut hasil pengujian semua endpoint:

POST http://localhost:8000/api/auth/register Send

Docs Params Authorization Headers (9) Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify


```
{
    "nama": "P. Calista",
    "email": "2410511141_cal@example.com",
    "password": "utsservice2026"
}
```


Body Cookies Headers (13) Test Results 201 Created 11.15 s 688 B

{} JSON Preview Visualize


```
{
    "status": "success",
    "message": "Registrasi berhasil",
    "data": {
        "id": 5,
        "nama": "P. Calista",
        "email": "2410511141_cal@example.com",
        "role": "user",
        "foto_profil": null,
        "oauth_provider": null,
        "dibuat_pada": "2026-05-01T04:14:31.000Z"
    }
}
```


POST http://localhost:8000/api/auth/login Send

Docs Params Authorization Headers (9) Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify


```
{
    "email": "2410511141_cal@example.com",
    "password": "utsservice2026"
}
```


Body Cookies Headers (13) Test Results 200 OK 609 ms 1.01 KB

{} JSON Preview Visualize


```
{
    "status": "success",
    "message": "Login berhasil",
    "data": {
        "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
            eyJ1c2VyX2lkIjo1LCJlbWFpbCI6IjI0MTA1MTExNDFfY2FsQGV4YW1wbGUuY29tIiwicm9sZSI6InVzZXIiLCJpYXQiOjE3Nzc2MTQ0NzQsImV4cCI6MT
            c3NzYxNTM3NH0.ZK8UPRvvnldEjHczzxVozsDJ9lNZMnPLezPsHgpQzB8",
        "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo1LCJpYXQiOjE3Nzc2MTQ0NzQsImV4cCI6MTc3ODIxOTI3NH0.
            e9WvWahs8l0VYbrz5gNzwN_Ht8bAsROryt5F0KtL75k",
        "token_type": "Bearer",
        "expires_in": 900,
        "user": {
            "id": 5,
            "nama": "P. Calista",
            "email": "2410511141_cal@example.com",
            "role": "user"
        }
    }
}
```


GET http://localhost:8000/api/events?page=1&per_page=2 Send

Docs Params Authorization Headers (7) Body Scripts Tests Settings Cookies

Body Cookies Headers (17) Test Results 200 OK 423 ms 1.98 KB

{} JSON Preview Visualize


```
    "status": "success",
    "data": [
        {
            "id": "1",
            "organizer_id": "1",
            "category_id": "1",
            "judul": "Konser One Direction Reunion",
            "deskripsi": "Konser reuni spektakuler One Direction",
            "lokasi": "GBK Jakarta",
            "tanggal_mulai": "2026-06-10 19:00:00",
            "tanggal_selesai": "2026-06-10 23:00:00",
            "banner_url": null,
            "status": "published",
            "dibuat_oleh": "1",
            "dibuat_pada": "2026-04-30 14:14:21",
            "diubah_pada": "2026-04-30 14:14:21",
            "nama_organizer": "Autobot Event Corp"
        },
        {
            "id": "2",
            "organizer_id": "2",
            "category_id": "2",
            "judul": "Seminar PJM Aspirasi",
            "deskripsi": "Seminar Pendidikan Jurnalistik Mahasiswa",
            "lokasi": "UPNVJ",
            "tanggal_mulai": "2026-07-11 08:00:00",
            "tanggal_selesai": "2026-07-12 16:00:00",
            "banner_url": null,
            "status": "published",
            "dibuat_oleh": "1",
            "dibuat_pada": "2026-04-30 14:14:21",
            "diubah_pada": "2026-04-30 14:14:21",
            "nama_organizer": "Decepticon Creative"
```

POST http://localhost:8000/api/tickets/order

Send

Docs Params Authorization Headers (10) Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
{
  "event_id": 3,
  "items": [
    {
      "category_id": "5",
      "jumlah": 1
    }
  ]
}
```

Body Cookies Headers (13) Test Results 201 Created 42.31 s 2.74 KB

{} JSON Preview Visualize


```
            {
                "category_id": "5",
                "nama_kategori": "Early Bird",
                "harga_satuan": "50000.00",
                "jumlah": 1,
                "subtotal": 50000,
                "tikets": [
                    {
                        "id": 11,
                        "kode_tiket": "TKT-CABA9695-68C",
                        "qr_code": "
iVBORw0KGgoAAAANSUhEUgAAASwAAAEsCAYAAAB5fY51AAAAAklEQVR4AewaftIAAAWBSURBVO3BQY4YBw4EsJIw//
+y1pccs0ADbrgrJjn3SwAKbABKbABKbABKbABKbABKbABKbABKbABKbABKbABKbABKbABKbABKbABKbABKbABKbABK
bABKbABKbABKbABK/ORlMxN+n7vLm2Ymb7q7fMnMhN/
n7vKmDUCJDUCJDUCJDUCJDUCJDUCJDUCJDUCJDUCJDUCJDUCJDUCJDUCJDUCJDUCJDUCJn3zM3eVvMjNpdnd5YmbyxN3l5
+4uf5OZyZdsAEpsAEpsAEpsAEpsAEpsAEpsAEpsAEpsAEpsAEpsAEpsAEpsAEr8pNzM5EvuLvCPmcmX3F2abQBKbABKbABKbABKbABKbABKbABKbABKbABKbABKbABK/AQ+bGbyxN2F/
64NQIkNQIkNQIkNQIkNQIkNQIkNQIkNQIkNQIkNQIkNQIkNQIkNQIkNQIkNQIkNQIkNQIkNQIkNQIkNQIkNQIkNQIkNQImfiLu
7809mJl9yd/mb3F34fTYAJTYAJTYAJTYAJTYAJTYAJTYAJTYAJTYAJTYAJTYAJX7yMTMT/py7yxMzkyfuLk/
MTJ64u7xpZsKfswEosQEosQEosQEosQEosQEosQEosQEosQEosQEosQEosQEosQEosQEosQEosQEosQEosQEosQEosQEosA
```


GET http://localhost:8000/api/tickets/order/6

Send

Docs Params Authorization Headers (7) Body Scripts Tests Settings Cookies

Headers 6 hidden

Key	Value	Description
Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.ey...	
Key	Value	Description

Bulk Edit Presets

Body Cookies Headers (13) Test Results 200 OK 50 ms 953 B

{} JSON Preview Visualize


```
{
    "status": "success",
    "data": {
        "id": 6,
        "kode_order": "ORD-1777610537353-CB982C3C",
        "user_id": 5,
        "event_id": 3,
        "total_harga": "50000.00",
        "status_order": "paid",
        "dibuat_pada": "2026-05-01T04:42:30.000Z",
        "diubah_pada": "2026-05-01T04:42:38.000Z",
        "items": [
            {
                "id": 6,
                "order_id": 6,
                "category_id": 5,
                "nama_kategori": "Early Bird",
                "harga_satuan": "50000.00",
                "jumlah": 1,
                "subtotal": "50000.00",
                "tikets": [
                    {
                        "id": 11,
                        "kode_tiket": "TKT-CABA9695-68C",
                        "status_tiket": "digunakan",
```


POST http://localhost:8000/api/tickets/validate

Send

Docs Params Authorization Headers (10) Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
{
  "kode_tiket": "TKT-690D0F28-888"
}
```

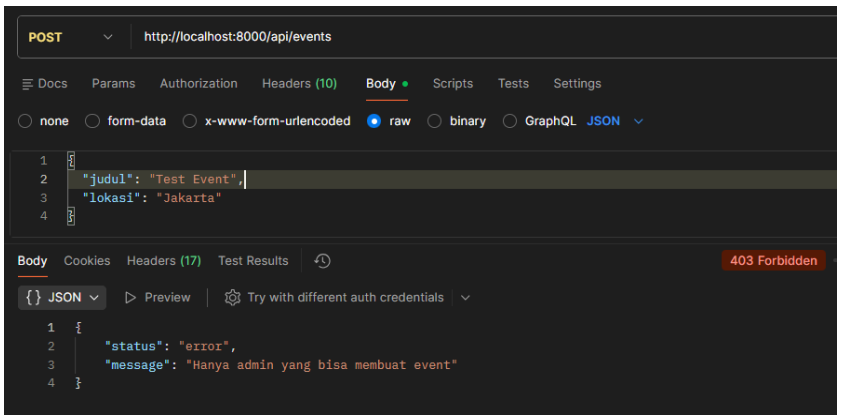
Body Cookies Headers (13) Test Results 200 OK 66 ms 627 B

{} JSON Preview Visualize


```
{
    "status": "success",
    "message": "Tiket valid! Silakan masuk.",
    "data": {
        "kode_tiket": "TKT-690D0F28-888",
        "nama_kategori": "Early Bird",
        "event_id": 3,
        "status": "digunakan"
    }
}
```

B. Pengujian Otorisasi Role

Pengujian dilakukan dengan mencoba mengakses endpoint admin menggunakan token user biasa. Sistem mengembalikan response 403 Forbidden sesuai yang diharapkan.



V. KESIMPULAN

Sistem pemesanan tiket event berbasis microservice berhasil dibangun dengan memenuhi requirement yang ditentukan. Sistem terdiri dari 3 microservice yang independen dan masing-masing memiliki database terpisah dan berkomunikasi melalui REST API Gateway.

Arsitektur microservice terbukti memberikan keuntungan dalam hal isolasi kegagalan, fleksibilitas teknologi per service, dan kemudahan scaling per komponen. Dibandingkan dengan pendekatan monolitik, arsitektur ini lebih kompleks di awal namun jauh lebih mudah dikembangkan dan dipelihara dalam jangka panjang.